

Dynamic Programming Series Striver

Topic House Robbery [Lc\(198\)](#)

Approach 1—> Recursion based Brute force

```
#include<iostream>
using namespace std;
int houserobbery(vector<int>&houses,int n){
    if(n<0)return 0;
    if (n==0)return houses[0];
    int pick=houses[n]+houserobbery(houses,n-2);
    int notpick=0+houserobbery(houses,n-1);
    return max(pick,notpick);
}
int main(){
    vector<int>houses;
    int n;
    cin>>n;
    int j;
    for(int i=0;i<n;i++){
        cin>>j;
        houses.push_back(j);
    }
    int maxmprofit=houserobbery(houses,n-1);
    cout<<maxmprofit<<endl;
    return 0;
}
```

Approach -2 —> Recursion using dp memomization

```
#include<iostream>
using namespace std;
int houserobbery(vector<int>&houses,int n,vector<int>&dp){
    if(n<0)return 0;
    if(n==0)return houses[n];

    if(dp[n]!=-1)return dp[n];
    int keep=houses[n]+houserobbery(houses,n-2,dp);
    int notkeep=0+houserobbery(houses,n-1,dp);
    return dp[n]=max(keep,notkeep);
}
int main(){
    vector<int>houses;
    int n;
    cin>>n;
    int j;
    vector<int>dp(n,-1);
    for(int i=0;i<n;i++){
        cin>>j;
        houses.push_back(j);
    }
    int maxmprofit=houserobbery(houses,n-1,dp);
    cout<<maxmprofit<<endl;
    return 0;
}
```

Approach 3 → Tabulation Method

```
#include<iostream>
using namespace std;

int houserobbery(vector<int>&houses,int n,vector<int>&dp){
    dp[0]=houses[0];
    for(int i=1;i<n;i++){
        int keep=houses[i];
        if(i-2<0)keep+=0;
        else keep+=dp[i-2];
        int notkeep=0+dp[i-1];
        dp[i]=max(keep,notkeep);
    }
    return dp[n-1];
}

int main(){
    vector<int>houses;
    int n;
    cin>>n;
    int j;
    vector<int>dp(n);
    for(int i=0;i<n;i++){
        cin>>j;
        houses.push_back(j);
    }
    int maxmprofit=houserobbery(houses,n,dp);
    cout<<maxmprofit<<endl;
    return 0;
}
```

Approach 4 → Optimal space computation

```
#include<iostream>
using namespace std;
int houserobbery(vector<int>&houses,int n,vector<int>&dp){
    int p2=0;
    int p1=houses[0];
    for(int i=1;i<n;i++){
        int keep=houses[i];
        if(i-2>=0)keep+=p2;
        int notkeep=p1;
        int curr=max(keep,notkeep);
        p2=p1;
        p1=curr;
    }
    return p1;
}
int main(){
    vector<int>houses;
    int n;
    cin>>n;
    int j;
    vector<int>dp(n);
    for(int i=0;i<n;i++){
        cin>>j;
        houses.push_back(j);
    }
    int maxmprofit=houserobbery(houses,n,dp);
    cout<<maxmprofit<<endl;
    return 0;
}
```

Time Complexity	Space Complexity
$O(2^n)$	$n + \log n$
$O(n)$	$O(n)$
$O(n)$	$O(n)$
$O(n)$	$O(1)$